

Module 5 Project Rubric

PromptLab | Deployed Full-Stack Application | AIE 500 - 10x Coding with AI

C3.1-C3.5 | Application evidence here; C3.6-C3.8 follow in Module 6.

Complete the remaining backend feature test-first, build React from a frontend specification and deliver a reproducible full-stack PromptLab application.

Evidence to prepare

- specs/frontend.md committed before the React implementation, with a component structure that follows the specification.
- The second specified feature and any missing backend endpoints built test-first; test results against the deployed configuration.
- Both version history and tagging present in the final application, with real API core journeys and loading, empty and visible error states.
- A deployed application or documented full-stack container run, plus docs/deployment.md covering every step, variable and secrets-handling approach.
- Honest timestamps for scaffolding to the first passing backend endpoint, plus the advance plan and any blockers. See the C3.5 timing note on page 3.

MUST PASS: C3.3 - Your app runs end to end

A Not Yet on this requirement returns the submission without reviewing the remaining criteria in this module.

How grading works

Every criterion is **Met** or **Not Yet**. There are no points, percentages, weights or bonus credit. All criteria in a competency must be Met; nothing is averaged.

Three attempts are available per module evidence group. Returned submissions do not consume an attempt, but require a revision plan. Re-assessment covers Not Yet criteria and any evidence affected by the changes. Follow the Syllabus for policy and course instructions for submission timing.

Returned before assessment: Missing deliverables, failing provided tests, an application that does not run or committed secrets. Rotate exposed credentials before resubmitting. Work belonging to another person or reconstructed logs/history are referred for academic review.

Module 5 | Assessment criteria (1/2)

C3 - End-to-End AI-Assisted Application Delivery

C3.1-C3.5 | Application evidence here; C3.6-C3.8 follow in Module 6.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C3.1 Your structure follows your spec	specs/frontend.md, frontend/src/	Every component in your frontend spec exists as its own module, and you can state the organizing principle of your folder structure in one sentence. Backend and frontend concerns stay separate.	- Vite template directories left in unused. - Specified components missing or merged with no explanation. - Concerns tangled across modules.
C3.2 Your backend increment was built test-first	Commit history for your Module 5 feature; test run against deployed config	The second spec feature, and any endpoints your frontend needs, show a failing test before implementation in commit history. The suite passes against your deployed configuration, not just locally.	- Tests added once the feature was finished. - A suite that passes locally but not against the deployed config. - Tests that exercise FastAPI rather than your own logic.
C3.3 Your app runs end to end (MUST PASS)	Deployed app or documented container run; frontend/, backend/	The deployed application completes its core journeys: list prompts, view detail, create, edit, delete with confirmation, manage collections, filter by collection, search. The frontend consumes your real API. Loading and empty states present, and API failures produce visible user-facing messages.	- A frontend running on mocked or hard-coded data. - A core journey breaking partway through. - API errors producing a blank screen or nothing at all.

Core journeys must use the real backend. Check list, detail, create, edit, confirmed delete, collection management, collection filtering and search in the documented deployment configuration. A generated UI, mocks or a green backend suite alone cannot establish C3.3.

Module 5 | Assessment criteria (2/2)

C3 - End-to-End AI-Assisted Application Delivery

C3.1-C3.5 | Application evidence here; C3.6-C3.8 follow in Module 6.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C3.4 Someone else could deploy it	docs/deployment.md, committed configuration	A second person can deploy or run PromptLab using only what's committed. Every step, environment variable and secrets-handling approach is documented.	- Deployment relying on something you configured by hand and never wrote down. - Instructions missing environment variables. - Only a running instance, with no reproducible path to it.
C3.5 You worked inside the time budget	Commit timestamps for the Module 5 backend increment	Scaffolding to first passing endpoint completed within the budget stated in this module's tasks, shown by commit timestamps.	- Going over with no blocker documented. - Committing everything in one batch, so timestamps prove nothing. - Committing unfinished work to hit the time and finishing later.

C3.5 timing: calibration required

The original project brief gives four hours for this increment. The instructor handbook identifies that number as an unvalidated placeholder: collect real commit timestamps, but do not use it for a timing-based Not Yet until pilot calibration, a published budget and criterion-version update. Pending calibration is not an automatic Met. This interval is not a deadline for building the entire application.

Reproducible means another person can run it. A documented container route is accepted by the source rubric; a paid hosting service is not required. Keep committed configuration and instructions consistent with the environment you actually verified.

Source: PromptLab Grading Rubric v2.1. Original Modules 1-5 map to current Modules 2-6. Criterion standards are retained; module/tool references are updated. The Syllabus governs policy.